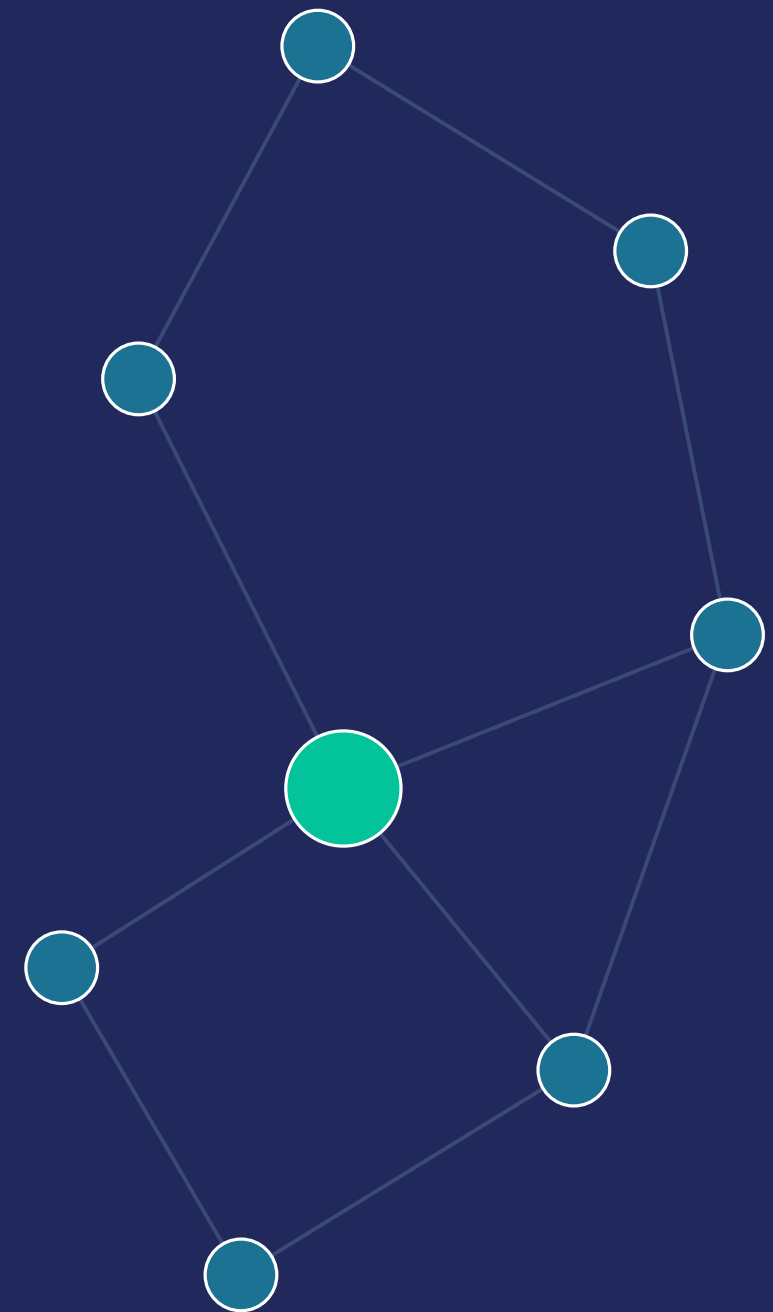


DATA SCIENCE · MACHINE LEARNING

Feature Selection in Machine Learning

Choosing the right variables for faster, simpler, more accurate models

Machine Learning & Data Analysis Course
Abdulrahman Bakeer



Learning Path

1

Introduction to Feature Selection

Motivation, definition, workflow, and categories

2

Filter Methods

Statistical ranking with Chi-Square test focus

3

Wrapper Methods

Model-based iterative feature selection approaches

4

Embedded Methods

Integrated selection with Gradient Boosting focus



Systematic Learning

Feature selection techniques

01

Introduction to Feature Selection

Understanding the fundamentals:

why, what, and how of feature selection in ML pipelines

The Motivation Behind Feature Selection



Curse of Dimensionality

As features grow, data becomes sparse and distance metrics lose meaning



Noise Reduction

Removing irrelevant features prevents models from learning random patterns



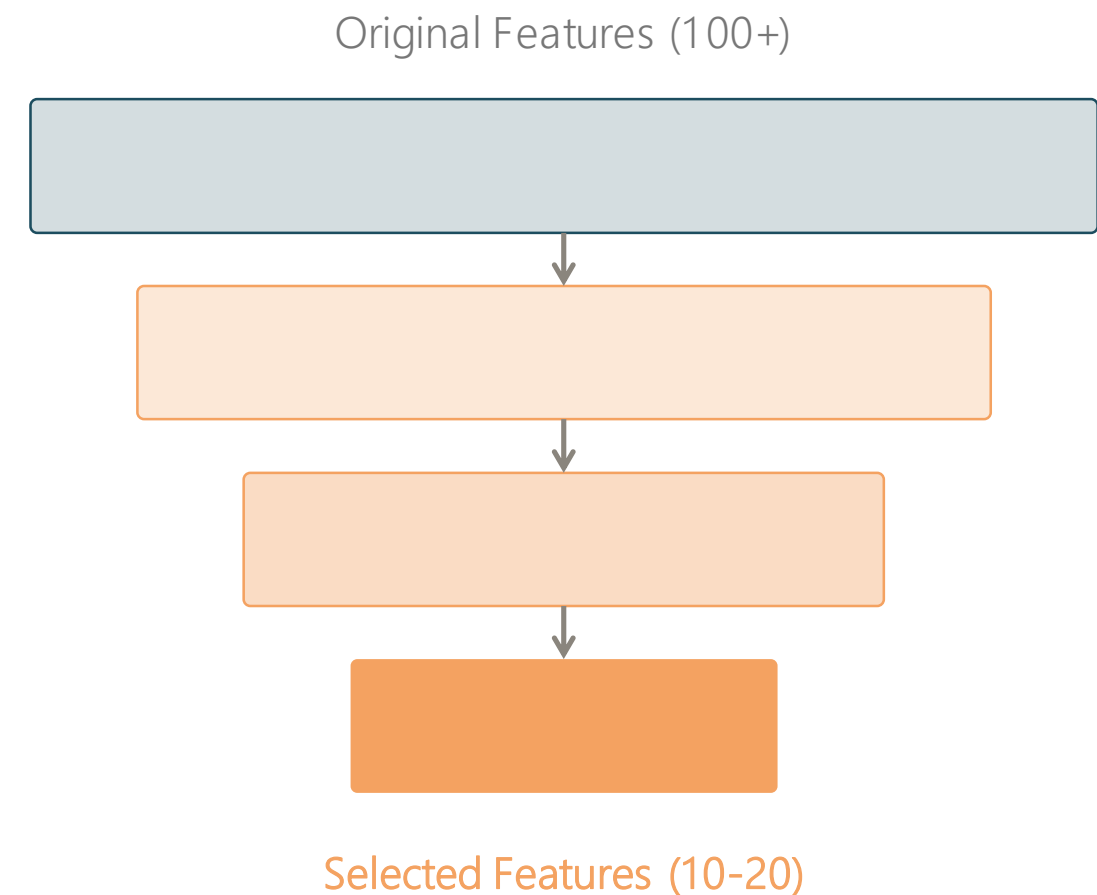
Faster Training

Fewer features mean less computation and significantly shorter training times



Better Generalization

Simpler models with relevant features perform better on unseen data



Key Insight: More features $\neq$ Better performance. The goal is to find the minimal set of features that maximizes predictive power.

What is Feature Selection?

Definition: The process of selecting a subset of relevant features for use in model construction, removing redundant or irrelevant variables.

Feature Selection



- Selects a **subset** of original features
- Preserves **interpretability**
- Feature names remain meaningful

Example:

From {age, income, height, zip} → select {age, income}

VS

Feature Extraction



- Creates **new features** from originals
- Transforms the **representation**
- May lose direct interpretability

Example:

PCA, LDA — create synthetic components

Why Feature Selection Matters



Accuracy

Removing noisy or irrelevant features improves predictive performance by helping the model focus on meaningful patterns in the data.



Interpretability

Fewer features make models easier to understand, explain, and debug — critical for domains like healthcare and finance.



Computation Cost

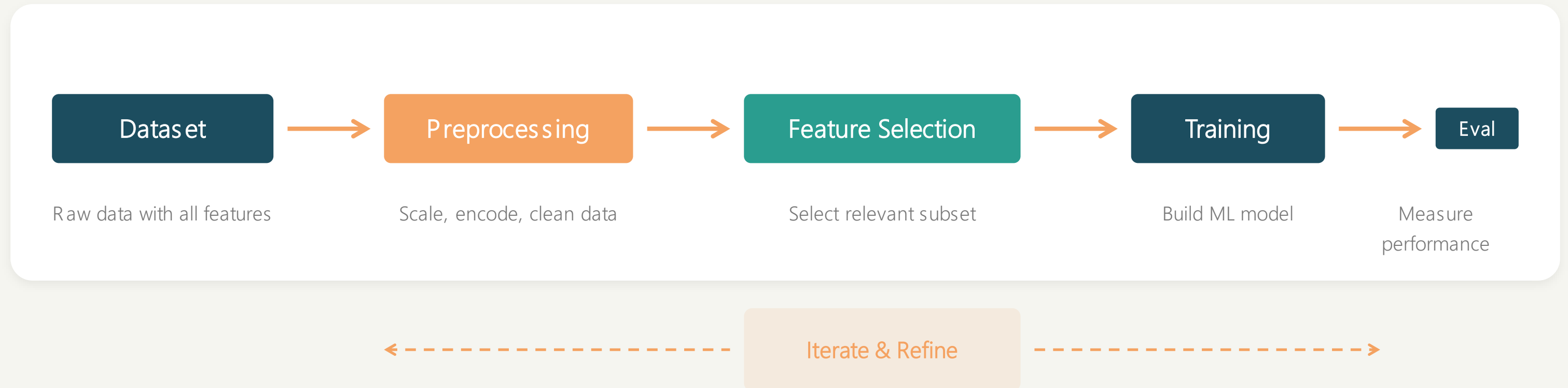
Reduces training time and memory requirements significantly — especially important for large-scale applications.



Overfitting Prevention

Prevents models from memorizing noise in high-dimensional data, leading to better performance on unseen data.

General Feature Selection Workflow



Important Ordering

Feature selection must happen **after** preprocessing but **before** model training. The feedback loop allows iterative refinement — if evaluation shows poor performance, you may need to revisit your feature selection strategy.

Categories of Feature Selection

Filter Methods

Model Dependency

Independent of ML algorithm

Speed

Fast — single pass statistics

Computational Cost

Low

Examples

- Chi-Square Test
- Correlation Coefficient
- Information Gain
- ANOVA F-test

Best For

Initial screening, large datasets

Wrapper Methods

Model Dependency

Uses model performance directly

Speed

Slow — iterative retraining

Computational Cost

High

Examples

- Recursive Feature Elimination
- Forward Selection
- Backward Elimination

Best For

Small-medium datasets, accuracy-critical

Embedded Methods

Model Dependency

Integrated during training

Speed

Moderate — single training pass

Computational Cost

Moderate

Examples

- Lasso (L1 Regularization)
- Tree-based Methods
- Gradient Boosting

Best For

General purpose, balanced approach

02

Filter Methods

Statistical techniques that rank features
independently of any machine learning algorithm

Filter Methods Overview

- Rank features using statistical tests, independent of any ML model
- “Model-independent” — score is computed before any model is trained
- Fast, scalable pre-processing step, especially for high-dimensional data
- Common tests: correlation, chi-square, information gain, variance

Typical Workflow

1. Compute a statistical score per feature



2. Rank features by score



3. Keep top-k or above-threshold features



4. Pass selected features to any model

ADVANTAGES

- Very fast, low computational cost
- Scales to thousands of features
- Works with any downstream model

LIMITATIONS

- Ignores feature interactions
- No feedback from actual model performance
- Threshold / k choice can be arbitrary

Comparison Diagram



Information Gain

- Entropy: measures impurity / disorder in the target labels
- Information Gain: reduction in entropy after splitting on a feature
- Higher IG → feature separates classes more cleanly
- Works well for classification, especially with categorical or discretized features

Entropy

$$H(S) = -\sum p_i \log_2(p_i)$$

Information Gain

$$IG(S,A) = H(S) - \sum (|S_v|/|S|) \cdot H(S_v)$$

ADVANTAGES

- Captures non-linear relevance
- Intuitive, used inside decision trees

LIMITATIONS

- Biased toward high-cardinality features
- Needs binning for continuous data

```
information_gain.py

from sklearn.feature_selection import mutual_info_classif
from sklearn.feature_selection import SelectKBest

# score each feature's mutual info with the target
scores = mutual_info_classif(X_train, y_train)

# keep the 10 features with highest information gain
selector = SelectKBest(mutual_info_classif, k=10)
X_new = selector.fit_transform(X_train, y_train)

selected = X_train.columns[selector.get_support()]
print(selected)
```

HOW IT WORKS

- mutual_info_classif() scores each feature vs. the target
- SelectKBest(k=10) keeps only the top-10 scoring features
- get_support() returns a boolean mask of the kept columns

Chi-Square Test for Feature Selection

Concept & Purpose

A statistical test to determine if there's a significant association between a **categorical feature** and the **categorical target** variable.

Measures how much observed frequencies deviate from expected frequencies under independence.

Variable Explanation

O = Observed frequency in contingency table

E = (row total × column total) / grand total

df = (rows - 1) × (columns - 1)

Chi-Square Formula

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Interpretation

- ▶ Higher χ^2 value → stronger association between feature and target
- ▶ Compare with critical value from χ^2 distribution table
- ▶ If $\chi^2 > \text{critical value}$ → feature is **relevant**



Remember: Chi-Square only works with categorical variables. For continuous features, use correlation instead

Chi-Square Test for Feature Selection

Chi-Square Formula

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Interpretation

- ▶ Higher χ^2 value → stronger association between feature and target
- ▶ Compare with critical value from χ^2 distribution table
- ▶ If $\chi^2 >$ critical value → feature is **relevant**

```
chi_square.py

from sklearn.feature_selection import SelectKBest, chi2

# keep the 8 features most associated with y
selector = SelectKBest(score_func=chi2, k=8)
X_new = selector.fit_transform(X_train, y_train)
```

ADVANTAGES

- Simple, well-understood statistic
- Fast even on wide, categorical data

LIMITATIONS

- Only for categorical / non-negative data
- Ignores feature interactions

HOW IT WORKS

- chi2 computes the χ^2 statistic between each feature and y
- SelectKBest(k=8) ranks and keeps the top-8 features
- X_train must be ≥ 0 — scale or encode features first

Chi-Square: Step-by-Step Example

1 Observed Frequencies

Weather	Play=Yes	Play=No	Total
Sunny	3	2	5
Rainy	2	3	5

2 Expected Frequencies

Weather	Play=Yes	Play=No	Formula
Sunny	2.5	2.5	$(5 \times 5) / 10$
Rainy	2.5	2.5	$(5 \times 5) / 10$

3 Chi-Square Calculation

$$\chi^2(\text{Sunny, Yes}) = (3 - 2.5)^2 / 2.5 = 0.10$$

$$\chi^2(\text{Sunny, No}) = (2 - 2.5)^2 / 2.5 = 0.10$$

$$\chi^2(\text{Rainy, Yes}) = (2 - 2.5)^2 / 2.5 = 0.10$$

$$\chi^2(\text{Rainy, No}) = (3 - 2.5)^2 / 2.5 = 0.10$$

4 Interpretation

$$\text{Total } \chi^2 = 0.40$$

$$\text{df} = (2 - 1) \times (2 - 1) = 1$$

$$\text{Critical value } (\alpha = 0.05) = 3.841$$



Result: Since $0.40 < 3.841$, we fail to reject independence. In this toy example, Weather shows weak association with Play. With larger datasets, differences become more significant.

Chi-Square: Advantages & Limitations

Advantages

- ✓ Simple to understand and implement
- ✓ Computationally efficient — $O(n)$
- ✓ Works well with categorical data
- ✓ Provides statistical significance measure
- ✓ Model-independent — works with any classifier
- ✓ Good for initial feature screening

Limitations

- ✗ Only works with categorical features
- ✗ Assumes feature independence
- ✗ Misses features important only in combination
- ✗ Sensitive to small sample sizes
- ✗ Cannot capture non-linear relationships
- ✗ Requires expected frequency ≥ 5 per cell



Best Practice: Use Chi-Square as an initial screening tool, then validate selected features with wrapper or embedded methods for your specific model.

Fisher Score · Pearson Correlation · Variance Threshold

1 Fisher Score

$$F(x) = \sum n_i (\mu_i - \mu)^2 / \sum n_i \sigma_i^2$$

Ranks features by between-class vs within-class variance. Use for classification with numeric features.

code

```
from skfeature.function.similarity_based
import fisher_score

scores = fisher_score.fisher_score(X, y)
ranked = (-scores).argsort()
```

2 Pearson Correlation

$$r = \text{cov}(X, Y) / (\sigma_x \sigma_y)$$

Measures linear association with a numeric target. Use to drop weakly-correlated or redundant features.

code

```
import pandas as pd

corr = df.corr()['target'].abs()
top = corr.sort_values(ascending=False)
keep = top[top > 0.3].index
```

3 Variance Threshold

$$\text{Var}(X) < \tau \rightarrow \text{drop feature}$$

Removes near-constant features that carry almost no information, regardless of the target.

code

```
from sklearn.feature_selection import (
    VarianceThreshold)

vt = VarianceThreshold(threshold=0.01)
X_new = vt.fit_transform(X_train)
```

Mean Absolute Difference & Dispersion Ratio

Mean Absolute Difference (MAD)

- Average absolute deviation of each value from the feature mean
- Higher MAD → feature varies more → potentially more informative
- Cheap alternative to variance; less sensitive to large outliers

Formula

$$MAD = (1/n) \sum |x_i - \bar{x}|$$

mean_absolute_difference.py

```
import numpy as np

def mad(column):
    return np.mean(np.abs(column - column.mean()))

# score every numeric column
mad_scores = X_train.apply(mad)
top_features = mad_scores.sort_values(
    ascending=False).head(10).index
```

Dispersion Ratio

- Ratio of arithmetic mean (AM) to geometric mean (GM) of a feature
- Ratio near 1 → values are similar → low information
- Larger ratio → greater spread → feature likely more useful

Formula

$$DR = AM(x) / GM(x), \quad GM = (\prod x_i)^{(1/n)}$$

dispersion_ratio.py

```
import numpy as np

def dispersion_ratio(column):
    col = column[column > 0] # GM needs > 0
    am = col.mean()
    gm = np.exp(np.log(col).mean())
    return am / gm

dr_scores = X_train.apply(dispersion_ratio)
```

03

Wrapper Methods

Model-based iterative approaches

that use predictive performance to guide selection

Wrapper Methods: RFE, Forward & Backward Selection



Recursive Feature Elimination

1. Start with **all features**
2. Train model, rank features
3. Remove weakest feature(s)
4. Repeat until target count

Best for: When you know desired feature count



Forward Selection

1. Start with **no features**
2. Test adding each feature
3. Add the one that improves performance most
4. Repeat until stopping criterion

Best for: When few features are expected

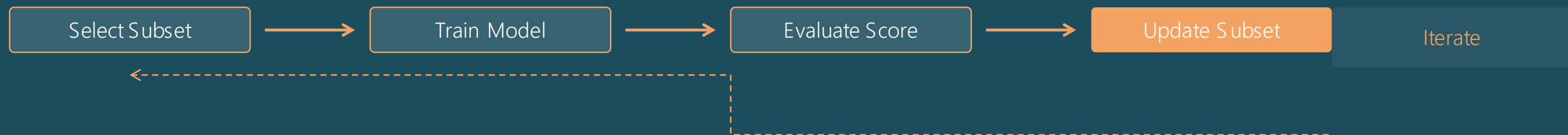


Backward Elimination

1. Start with **all features**
2. Test removing each feature
3. Remove the least significant
4. Repeat until all remaining are significant

Best for: When most features are relevant

Common Workflow Pattern



Wrapper Methods Overview

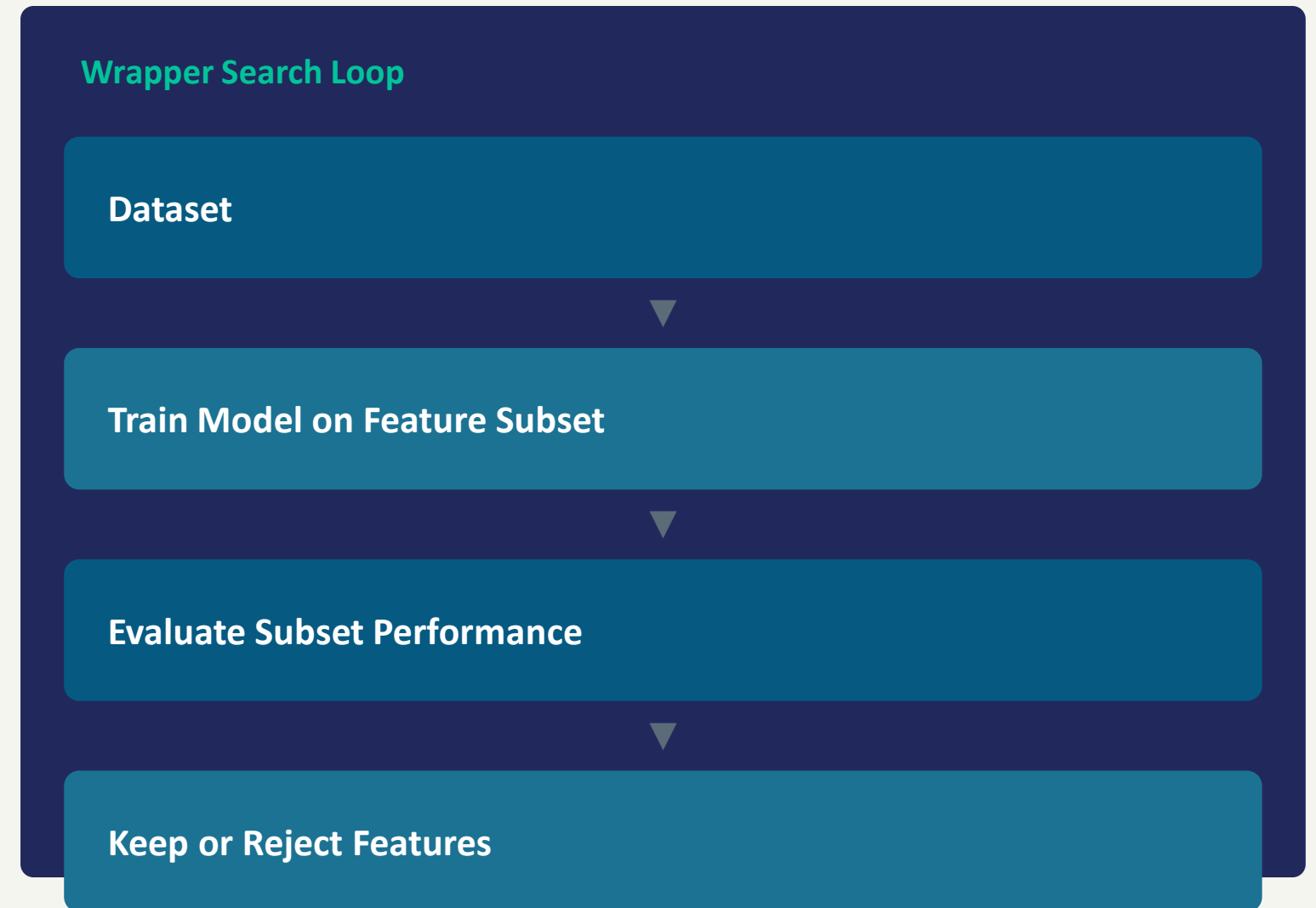
- Search for feature subsets by repeatedly training and evaluating a model
- Filter = statistics only; Wrapper = actual model performance as the score
- Explores feature interactions the model itself would see
- Cost scales with dataset size × number of subsets tried

ADVANTAGES

- Accounts for feature interactions
- Optimized directly for chosen model

LIMITATIONS

- Computationally expensive
- Risk of overfitting to validation set



Forward Selection

- Starts with an empty feature set
- Adds the single feature that most improves model performance
- Repeats until no further improvement (or target count reached)
- Good when few features are truly relevant, out of many candidates

Step-by-Step

1. Start with 0 features

2. Test adding each remaining feature

3. Permanently add the best performer

4. Repeat until stop condition is met

forward_selection.py

```
from sklearn.feature_selection import (
    SequentialFeatureSelector)
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

sfs = SequentialFeatureSelector(
    model, n_features_to_select=8,
    direction='forward', cv=5)

sfs.fit(X_train, y_train)
X_new = sfs.transform(X_train)
```

HOW IT WORKS

- direction='forward' grows the subset one feature at a time
- cv=5 scores each candidate with 5-fold cross-validation
- n_features_to_select fixes how many features to end with

Backward Elimination

- Starts with ALL features included
- Repeatedly removes the least useful feature (by score or p-value)
- Stops when removing further hurts performance
- Computationally heavier than forward selection on large feature sets

ADVANTAGES

- Considers full context of all features
- Often finds strong subsets

LIMITATIONS

- Expensive: many models trained early on
- Slow when the initial feature count is large

backward_elimination.py

```
from sklearn.feature_selection import (
    SequentialFeatureSelector)
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
sbe = SequentialFeatureSelector(
    model, n_features_to_select=8,
    direction='backward', cv=5)

sbe.fit(X_train, y_train)
```

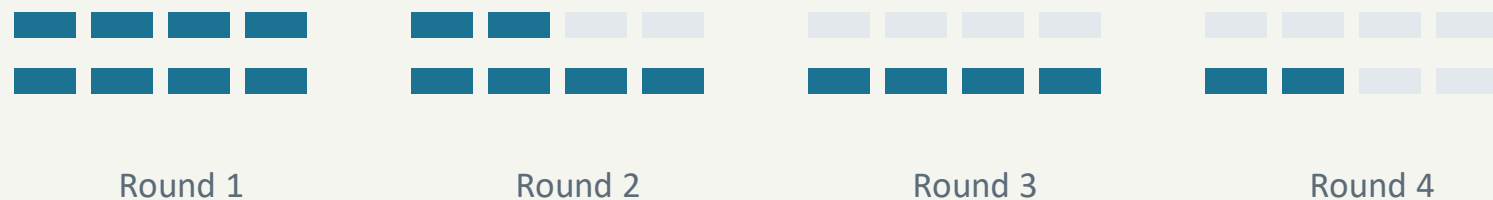
HOW IT WORKS

- direction='backward' starts full and removes one feature per round
- Each round drops whichever feature costs the model least
- statsmodels alternative: drop the feature with the highest p-value
- above 0.05 and refit, repeating until all p-values are significant

Recursive Feature Elimination (RFE)

- Trains the model, ranks features by importance / coefficient weight
- Removes the weakest feature(s), then retrain — recursively
- Continues until the target number of features remains
- Produces a full importance ranking, not just a kept/dropped label

Recursive Elimination



`rfe.py`

```
from sklearn.feature_selection import RFE
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)

rfe = RFE(estimator=model, n_features_to_select=6,
          step=1)
rfe.fit(X_train, y_train)

print(rfe.support_)    # kept features (bool mask)
print(rfe.ranking_)    # 1 = most important
```

HOW IT WORKS

- estimator provides feature importances / coefficients each round
- `step=1` removes one feature per iteration (higher = faster, coarser)
- `support_` is the final kept mask; `ranking_` orders every feature

Wrapper Methods: Trade-offs

The Accuracy vs. Cost Spectrum

Higher Accuracy

Higher Cost

Filter Methods

Embedded

Wrapper Methods

When to Use Wrapper Methods

- ✓ Small to medium datasets
- ✓ Model performance is critical
- ✓ Computational resources available
- ✓ Feature interactions are important
- ✓ Final model selection stage

When to Avoid Wrapper Methods

- ✗ Very large feature spaces
- ✗ Real-time applications
- ✗ Limited computational budget
- ✗ Need model-agnostic selection
- ✗ Quick prototyping phase

Comparison of All Methods

Method	Category	Uses a Model?	Cost	Best For
Information Gain	Filter	No	Low	Categorical / discretized data
Chi-Square	Filter	No	Low	Categorical classification
Fisher Score	Filter	No	Low	Numeric classification features
Pearson Correlation	Filter	No	Low	Linear, numeric relationships
Variance Threshold	Filter	No	Very Low	Removing near-constant features
Mean Absolute Diff.	Filter	No	Very Low	Quick spread-based screening
Dispersion Ratio	Filter	No	Low	Positive-valued numeric data
Forward Selection	Wrapper	Yes	High	Few relevant features expected
Backward Elimination	Wrapper	Yes	High	Smaller starting feature sets
Recursive Elim. (RFE)	Wrapper	Yes	High	Full importance ranking needed

Practical Guidelines: Which Method to Use?



High-dimensional dataset

Filter methods (fast, scale well)



Categorical features + target

Chi-Square, Information Gain



Small dataset

Wrapper methods (cost is manageable)



Numeric features, linear target

Pearson Correlation, Fisher Score



Need maximum accuracy

Wrapper methods / RFE



Highly correlated features

Correlation filtering, RFE



Need fast preprocessing

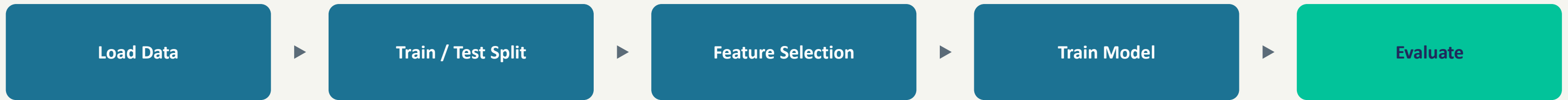
Variance Threshold, Correlation



Regression problems

Correlation, Wrapper with regression metric

Complete Python Workflow



`end_to_end_feature_selection.py`

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
# 1. Load a real, well-known dataset
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
```

```
# 2. Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
# 3. Select the 10 most informative features (filter method)
selector = SelectKBest(score_func=f_classif, k=10)
X_train_sel = selector.fit_transform(X_train, y_train)
X_test_sel = selector.transform(X_test)
```

```
# 4. Train a model on the reduced feature set
clf = RandomForestClassifier(random_state=42).fit(
    X_train_sel, y_train)
```

```
# 5. Evaluate on held-out test data
preds = clf.predict(X_test_sel)
print('Accuracy:', accuracy_score(y_test, preds))
```

04

Embedded Methods

Feature selection integrated directly
into the model training process

Introduction to Embedded Methods

Core Principle: Feature selection occurs *during* model training — the model itself decides which features are important.

Lasso (L1)

L1 regularization shrinks some coefficients to **exactly zero**, effectively selecting features.

The regularization parameter λ controls sparsity.

Tree-Based

Decision trees naturally compute **feature importance** by measuring impurity reduction at each split.

Random Forest aggregates importance across trees.

Gradient Boosting

Sequential ensemble that provides **built-in feature importance** scores based on gain and split frequency.

XGBoost, LightGBM, CatBoost.



Key Advantage: Embedded methods balance computational efficiency and model-specific optimization — faster than wrappers, more model-aware than filters.

Gradient Boosting: Core Concepts

Core Intuition

Build models **sequentially**, where each new model corrects errors made by the previous ensemble. Weak learners combine to form a strong predictor.

Key Parameters

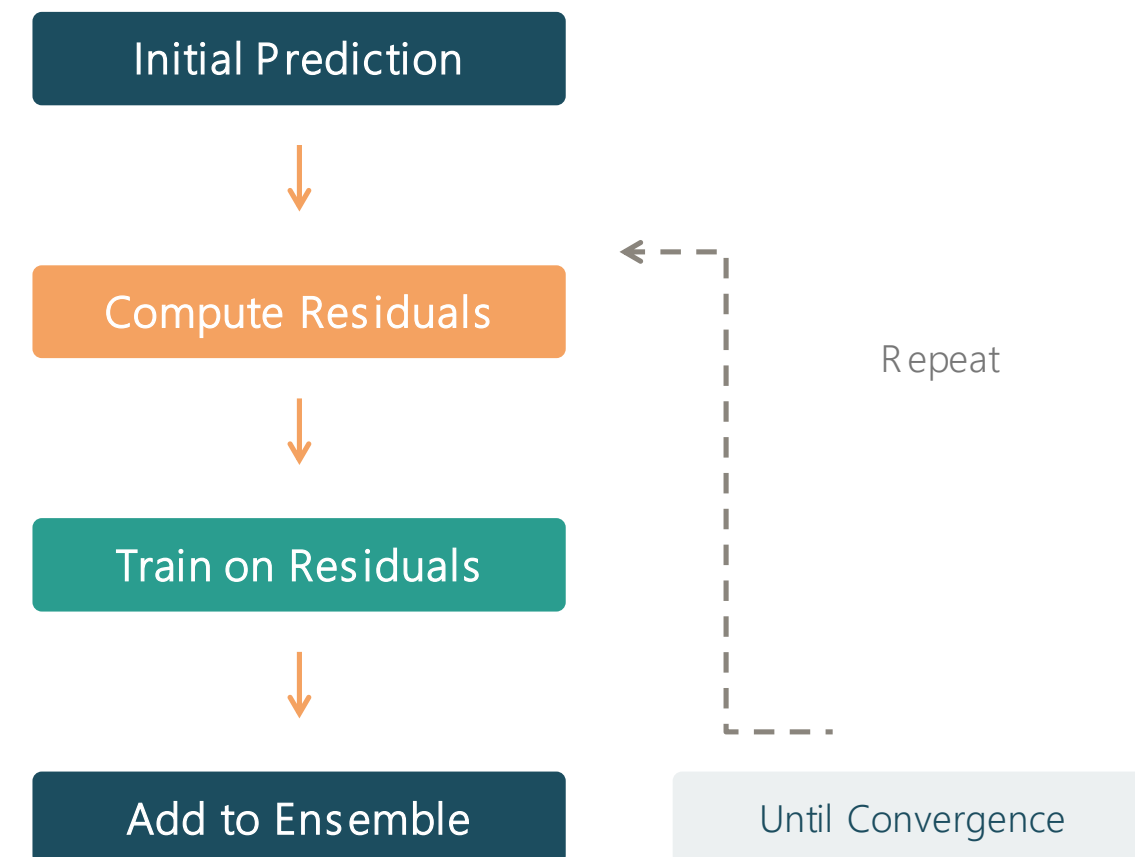
n_estimators — number of trees in ensemble

learning_rate — shrinkage factor (0.01–0.3)

max_depth — depth of each tree (3–6 typical)

subsample — fraction of data per tree

Boosting Flow Diagram



Key Idea: Each tree learns from the mistakes of all previous trees. The learning rate controls how aggressively we correct errors.

Feature Importance in Gradient Boosting

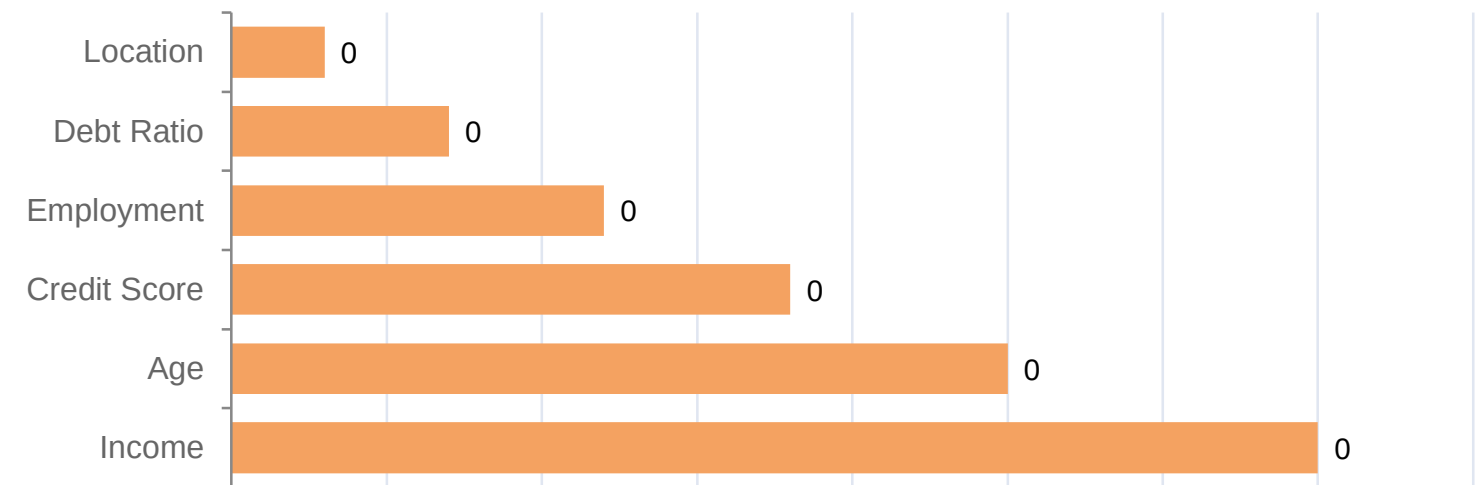
How Importance is Computed

Feature importance is based on how much each feature contributes to **reducing impurity** (Gini/Entropy) across all splits in all trees.

Two main metrics:

- **Gain** — total impurity reduction
- **Split Count** — frequency of use

Feature Importance Ranking (Example)



Feature Selection Process

Train Gradient
Boosting Model



Extract Feature
Importance Scores



Rank by Gain /
Split Count



Select Top-K

Note: Importance can be biased toward high-cardinality features. Consider permutation importance as an alternative for validation.

Gradient Boosting: Pros & Cons

Advantages

- ✓ High predictive performance
- ✓ Automatic feature importance ranking
- ✓ Handles mixed data types
- ✓ Robust to outliers
- ✓ Captures non-linear relationships
- ✓ Models feature interactions
- ✓ No need for feature scaling

Limitations

- ✗ Computationally expensive to train
- ✗ Risk of overfitting
- ✗ Less interpretable than single trees
- ✗ Sensitive to hyperparameters
- ✗ Sequential — hard to parallelize
- ✗ Can be biased to high-cardinality features
- ✗ Requires careful tuning



Pro Tip: Use early stopping and cross-validation to prevent overfitting. Libraries like XGBoost and LightGBM provide efficient implementations with built-in regularization.

Comparative Summary: Filter vs Wrapper vs Embedded

Dimension	● Filter	Wrapper	● Embedded
Speed	Fast	Slow	Moderate
Accuracy	Moderate	High	High
Complexity	Low	High	Moderate
Scalability	Excellent	Poor	Good
Model Dependency	None	High	Moderate
Feature Interactions	Ignored	Captured	Captured
Overfitting Risk	Low	High	Moderate
Best Use Case	Initial screening, large datasets	Small-medium datasets, accuracy-critical	General purpose, balanced approach



Recommendation: Start with Filter methods for quick screening, then refine with Embedded methods. Use Wrapper methods only when you have a small dataset and need maximum accuracy.

Real-World Applications



Medical Diagnosis

Selecting relevant **biomarkers** from thousands of genetic features to predict disease. Essential when samples are limited but feature space is vast.



Credit Scoring

Identifying key **financial indicators** that predict loan default risk. Regulatory requirements often demand explainable feature sets.



Computer Vision

Choosing discriminative features from **high-dimensional pixel data** for object recognition. Reduces noise after initial feature extraction.



Recommendation Systems

Selecting user behavior features that best predict **preferences and ratings**. Improves both model speed and recommendation quality.

Best Practices for Feature Selection

1 Avoid Data Leakage

Never use information from the **test set** during feature selection. Always fit selectors on **training data only**. Leakage leads to optimistically biased performance estimates.

2 Use Cross-Validation

Validate feature selection within each **CV fold** to ensure selected features generalize across different data splits. This prevents overfitting to a particular train/test split.

3 Leverage Domain Knowledge

Combine statistical methods with **subject matter expertise**. A weak statistical feature may be clinically or financially important. Domain knowledge improves interpretation.

4 Combine Methods

Use **filter methods** for initial screening to quickly eliminate obviously irrelevant features, then apply **wrapper or embedded** methods for final refinement.

Key Takeaways

- Feature selection is **essential** for efficient, interpretable, and generalizable ML models
- Three main approaches: **Filter** (fast, model-independent), **Wrapper** (accurate, expensive), **Embedded** (balanced)
- Choose based on **dataset size**, computational budget, and model requirements
- Always validate feature selection within **cross-validation** to prevent data leakage
- The best features are both **statistically significant** and meaningful in the problem domain

"The goal is not to have the most features, but the right features."

Thank You

Questions & Discussion

References

- [1] [Feature Selection Techniques in Machine Learning — GeeksforGeeks](#)
- [2] [Chi-Square Test — GeeksforGeeks](#)
- [3] [Gradient Boosting in Machine Learning — GeeksforGeeks](#)